

Documento de Arquitetura de Scripts e Assets - EPIPHANY (GASA)

Este documento define a estrutura técnica e a organização do projeto "Epiphany" dentro da Unity. Ele serve como um guia prático para a criação e gerenciamento de assets e scripts, visando otimizar o fluxo de trabalho e reduzir a frustração com a interface da Unity.

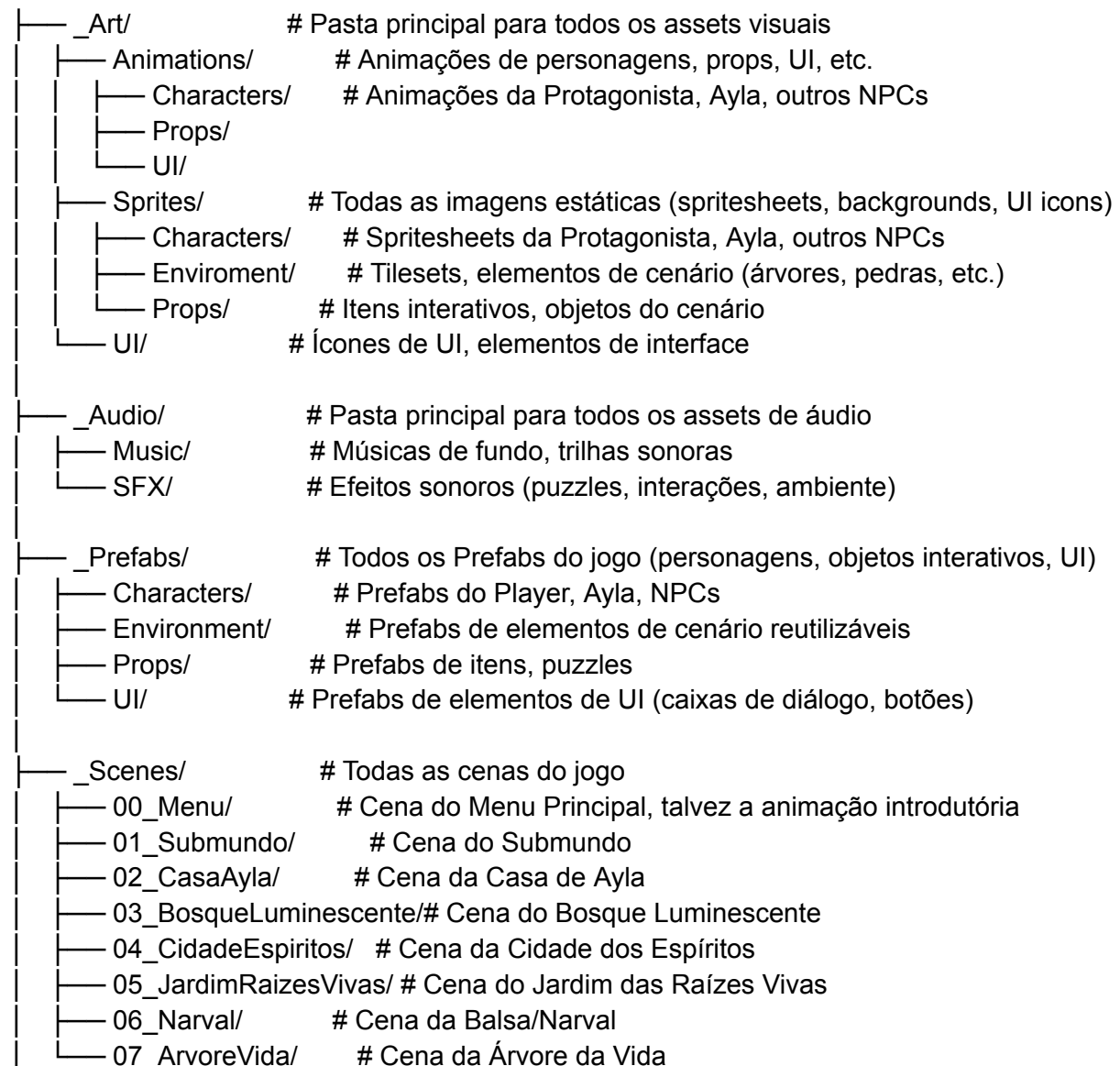
1. Estrutura de Pastas no Projeto Unity

Uma organização de pastas clara e consistente é a espinha dorsal de qualquer projeto Unity bem-sucedido. Ela facilita a navegação, a colaboração e a manutenção. A estrutura abaixo foi definida e implementada, seguindo as melhores práticas e adaptada às necessidades do projeto "Epiphany":

Princípio Geral: As pastas principais criadas pelo desenvolvedor utilizam o prefixo _ (underscore). Isso faz com que elas apareçam no topo da lista na janela Project da Unity, facilitando o acesso rápido aos arquivos mais importantes do projeto.

Estrutura Atualizada:

Assets/



```

|
|— _Scripts/          # Pasta principal para todos os scripts C# customizados
|   |— Core/          # Scripts fundamentais do jogo (ex: GameManager)
|   |— Data/          # Para ScriptableObjects que armazenam dados de jogo
|   |— Debug/         # Scripts temporários, de teste ou para depuração
|   |— Input/         # Assets e scripts relacionados ao Input System
|   |— Managers/      # Gerenciadores gerais (ex: AudioManager, SceneManager,
DialogueManager)
|       |— NPCs/      # Scripts de NPCs (ex: AylaController, SpiritLogic)
|       |— Player/    # Scripts relacionados ao Player (ex: PlayerController)
|       |— Puzzles/   # Scripts de lógica dos puzzles
|       |— ScriptableObjects/ # Definições de ScriptableObjects (DialogueData, ItemData,
CharacterData)
|           |— Tasks/  # Scripts de lógica das tasks
|           |— Transitions/ # Scripts para transições de cena/efeitos visuais (ex:
FadeController, CutsceneTransition)
|               |— UI/    # Scripts para a interface do usuário (ex: MenuPrincipalManager,
OptionsManager, SequentialTypewriter)
|                   |— Utilities/ # Funções utilitárias, extensões
|                   |— World/    # Scripts que controlam interações com elementos do cenário (ex:
DoorTrigger)
|
|— _Settings/          # Assets de configurações do jogo (Input Actions Asset, Physics
Settings)
|
|— _ThirdParty/        # Pastas para pacotes externos (TextMesh Pro, etc.)
|   |— TextMesh Pro/   # Conteúdo do pacote TextMesh Pro (se houver)
Use code with caution.

```

Observações e Justificativas:

Sua estrutura atual já estava muito próxima da ideal, com as pastas principais bem definidas.

Ajustes mínimos: Renomeei a cena 06_BalsaNarval para 06_Narval para consistência e clareza, e adicionei a pasta _Settings e _ThirdParty que são boas práticas para projetos Unity maiores ou que usam pacotes externos.

A pasta _Scripts/ScriptableObjects é onde você criará os tipos de ScriptableObjects (os arquivos .cs que definem a estrutura dos dados). Os assets criados a partir desses tipos (os arquivos .asset que contêm os dados reais) podem ser salvos em uma subpasta _Data/ dentro de Assets/, ou em subpastas mais específicas dentro de _Scripts/ScriptableObjects/Assets/ se preferir mantê-los mais perto de seus scripts. Por enquanto, a estrutura atual é ótima.

2. Arquitetura de Scripts

Esta seção define como seus scripts C# serão organizados, como eles se comunicarão entre si e quais padrões de design serão usados para garantir que o código seja limpo, modular, escalável e fácil de manter e depurar.

2.1. Comunicação entre Scripts (Como os Componentes Conversam)

Em um jogo Unity, diferentes GameObjects e seus scripts precisam "conversar" para que as mecânicas funcionem. Nosso objetivo é que essa comunicação seja clara e organizada, sem que um script precise saber muitos detalhes sobre o outro (desacoplamento).

Vamos usar duas formas principais de comunicação que são muito comuns na Unity e fáceis de aplicar:

a) Referências Diretas (Para Conexões Próximas e Fixas)

O que é: Quando um script "conhece" diretamente outro componente ou GameObject e pode chamar funções nele.

Quando usar: Para componentes no mesmo GameObject ou em GameObjects que são filhos diretos e que têm uma relação muito próxima e que não vai mudar muito.

Como fazer (na Unity UI): Usar campos public ou [SerializeField] nos scripts para arrastar e soltar a referência de outro GameObject ou componente no Inspector. [SerializeField] é o recomendado para manter a variável privada no código.

Exemplo Prático (PlayerController e Animator): No PlayerController, o _playerAnimator é uma referência direta configurada no Inspector, permitindo que o script controle as animações.

b) UnityEvents (Para Ações Acionadas no Inspector)

O que é: Um sistema que permite configurar, diretamente no Inspector, quais funções devem ser chamadas quando um evento específico acontece.

Quando usar: Ideal para cenários onde um objeto precisa "avisar" que algo aconteceu (ex: "A porta foi aberta", "O botão foi clicado"), e a "reação" pode ser configurada visualmente sem código extra. Muito comum para botões de UI e triggers (áreas que ativam algo).

Vantagens: Não precisa escrever código para conectar a "quem" vai ouvir; é visual e fácil de mudar.

Exemplo: Um DoorTrigger pode ter um UnityEvent OnDoorOpened que, configurado no Inspector, chama uma função no AudioManager para tocar um som de porta.

c) Eventos C# (Para Comunicação Mais Avançada no Código)

O que é: A forma mais flexível e poderosa de comunicação desacoplada em C#. Um script "publica" um evento, e qualquer outro script pode "assinar" para ser notificado.

Quando usar: Para comunicação mais complexa e global, onde muitos scripts podem precisar reagir a algo, mas o script que "avisa" não precisa saber de nenhum deles. Ex: GameManager avisando "Jogo Pausado".

Por que não vamos focar nisso agora: Embora poderoso, a configuração é mais no código puro e pode ser mais abstrata para iniciantes. Priorizaremos [SerializeField] e UnityEvents que têm um aspecto mais visual na Unity UI.

2.2. Padrões de Design de Scripts (Managers e Singletons)

Managers são scripts que controlam e coordenam subsistemas específicos do jogo. Para que esses Managers possam ser acessados por qualquer outro script no jogo, usamos o padrão Singleton.

O que são Managers? Scripts que controlam um subsistema (ex: GameManager para estado geral, SceneManager para cenas, AudioManager para áudio, DialogueManager para diálogos, MenuPrincipalManager para menus).

O Padrão Singleton: Garante que existe apenas uma única instância de uma classe (um único GameManager, por exemplo) e fornece um ponto de acesso global e fácil a ela (GameManager.Instance).

Vantagens: Acesso simplificado de qualquer script, evita duplicação e pode persistir entre cenas (com DontDestroyOnLoad).

Implementação em "Epiphany": Seu GameManager.cs já é um Singleton perfeitamente implementado. Ele cuida da persistência entre cenas e da lógica de spawn do jogador, sendo o coração central do jogo. Futuros Managers (como DialogueManager, AudioManager) também seguirão este padrão.

2.3. Uso de ScriptableObjects (Dados Separados da Lógica)

ScriptableObjects são "folhas de dados" que vivem no seu projeto Unity e contêm apenas dados, não lógica de jogo.

Problema que ele Resolve: Misturar dados (textos, configurações) com código nos scripts dificulta a edição e a reutilização.

Como Ajudam:

Separação: Dados em .asset files, lógica em .cs files.

Edição Fácil: Edite dados diretamente no Inspector, sem mexer no código.

Reutilização: Um ScriptableObject pode ser referenciado por múltiplos scripts/GameObjects.

Organização: Ficam organizados em pastas no Project window.

Casos de Uso em "Epiphany":

Diálogos e Reflexões Ambientais: Cada conversa, cada linha de reflexão pode ser um DialogueData ScriptableObject (conforme o script DialogueData.cs que você criou).

Puzzles: Configurações de puzzles (ex: sequência de luzes) podem ser armazenadas em PuzzleData ScriptableObjects.

Itens: Detalhes de itens (como a flauta) podem ser um ItemData ScriptableObject.

Status: Você já criou o script DialogueData.cs e o asset Submundo_AylaIntroDialogue.asset, demonstrando a compreensão e aplicação deste conceito.

2.4. Configuração do Input System (Novo) - Adaptando o PlayerController

O novo Input System da Unity oferece uma forma flexível e poderosa de gerenciar os controles do jogador.

Objetivo: Configurar um Input Actions Asset que contenha todas as ações de controle do jogador e conectar o PlayerController a ele.

Ações Definidas:

Gameplay Action Map:

Move (Value, Vector2): Para movimentação top-down (WASD ou Setas).

Interact (Button): Para interação com objetos/personagens (Tecla E).

AdvanceDialogue (Button): Para avançar diálogos ou menus (Tecla Enter ou Espaço).

OpenMenu (Button): Para abrir o menu de pausa (Tecla Esc).

OpenMap (Button): Para abrir o mapa (Tecla F).

Implementação no PlayerController.cs:

Adicionada a referência `private PlayerInputActions playerInputActions;`

Inicialização em `Awake()`: `playerInputActions = new PlayerInputActions();`

Habilitação/Desabilitação em `OnEnable()/OnDisable()`:
`playerInputActions.Gameplay.Enable()` e `playerInputActions.Gameplay.Disable()`.

Leitura da movimentação em Update(): `_playerDirection = playerInputActions.Gameplay.Move.ReadValue<Vector2>()`;
Assinatura de eventos para ações de botão em Awake() e desassinatura em OnDisable(): `playerInputActions.Gameplay.Interact.performed += OnInteractPerformed`; etc.
Criação de métodos de tratamento para cada ação de botão (ex: `OnInteractPerformed`).
Status: A configuração do `PlayerInputActions` e a adaptação do `PlayerController` para utilizá-lo foram concluídas com sucesso e testadas.

2.5. Arquitetura de Animação (Conectando o Movimento ao Visual)

A arquitetura de animação garante que as animações do personagem se integrem perfeitamente com a lógica de movimento e o Animator da Unity.

Componente Animator: Anexado ao `GameObject` do personagem, usa um `Animator Controller` para definir a lógica de reprodução das animações.

Parâmetros do Animator:

Float `MoveX`: Recebe o valor X do input de movimento.

Float `MoveY`: Recebe o valor Y do input de movimento.

Int `MovementState`: Recebe um valor inteiro que representa o estado geral de movimento (0: ParadoBaixo, 1: Correndo, 2: Cima, 3: Baixo, 4: ParadoCima).

Blend Tree 2D Freeform Directional:

Utilizado no `Animator Controller` para misturar animações de movimento (andar para cima, baixo, esquerda, direita) com base nos parâmetros `MoveX` e `MoveY`.

Permite a criação de animações suaves para todas as direções (incluindo diagonais) usando um número limitado de clips de animação.

Seu setup de ter uma animação para "andar para cima", "andar para baixo" e uma única para "andar para o lado" (para esquerda e direita com flip) é totalmente compatível e eficiente com este tipo de Blend Tree.

Conexão com `PlayerController`: O script `PlayerController.cs` envia os valores dos parâmetros (`MovementState`, `MoveX`, `MoveY`) para o `Animator` em cada `Update()`.

Status: A configuração do `Animator Controller` com o Blend Tree e a conexão com o `PlayerController` foram concluídas com sucesso e testadas.